Guion de demo — 10 minutos

Plataforma de Logistica Internacional y Monitoreo IoT · TP Persistencia Poliglota

Siete paradas que tocan **los cinco motores** y muestran lo mejor del trabajo. Cada una dice: que tipear, que va a salir, y **que decir mientras sale**.

> **Regla de oro:** no improvises con datos. Los IDs y codigos de este guion estan verificados. Si el profesor pide otra cosa, buscala con una opcion de listado primero y despues usala.

Antes de empezar (5 minutos antes)

```
cd C:\Users\ezeal\source\tp_logistica
.\.venv\Scripts\Activate.ps1
python scripts/preparar_demo.py
```

El script levanta las cinco bases, espera a que respondan y avisa si algo falla. **Lee el paso 5:** si dice que hay pocas posiciones GPS, corre esto antes de mostrar nada:

```
python -m logistica.jobs.simulador_iot --ciclos 30 --intervalo 0
```

Por que hace falta: las fechas de PostgreSQL son relativas al dia de carga, pero las lecturas de Cassandra llevan la fecha real en que se generaron. Los datos IoT envejecen; el simulador los refresca.

Despues:

```
python -m logistica
```

Entras con `admin@logistica.com` / `demo1234`.

> **Si Docker falla con "port is already allocated":** el script te dice que contenedor tiene el puerto y el comando para pararlo. En esta maquina suele ser `docker stop statsline-postgres`.

El recorrido

1. La puerta de entrada

1 min Redis + PostgreSQL

Ya entraste como admin. Antes de tocar nada, haz notar lo que se ve en pantalla:

> **Decir:** "La aplicacion no arma ningun menu hasta que alguien se autentica. El rol determina que secciones ve: yo soy administrador y veo las 16."

Despues sali y entra como `mariana@agroglobal.com`:

> **Decir:** "Mariana es del rol CLIENTE. Autentica correctamente y el sistema le dice que su rol no tiene acceso a ninguna seccion. La consola es una herramienta interna: un cliente no opera el back-office. **Esto es la prueba de que el control existe**, no una pantalla que dice que existe."

Si pregunta como: la sesion vive en Redis con TTL, porque es el unico dato del sistema que nace con fecha de muerte. Las contrasenas son PBKDF2 con 200.000 iteraciones y salt por usuario.

2. Facturacion: por que hace falta un relacional

1,5 min PostgreSQL

Volve a entrar como admin.

```
Menu 7 (Facturacion) → opcion 1: Listar cuentas corrientes
```

Sale una tabla con los tres tipos de titular y sus saldos.

> **Decir:** "Esto es lo que justifica PostgreSQL. Emitir una factura escribe tres cosas: la factura, el movimiento en el libro mayor y el saldo de la cuenta. **O pasan las tres o ninguna.** Eso es una transaccion, y es lo unico que ninguna de las otras cuatro bases garantiza."

```
Menu 7 → opcion 9: Verificar saldos (integridad)
```

> **Decir:** "El saldo esta guardado ademas del libro mayor, que es una denormalizacion a proposito: derivarlo sumando todos los movimientos crece sin limite. El riesgo de que se desincronicen lo controla esto, que compara el saldo guardado contra la suma de los movimientos."

3. Recorrido GPS: la serie temporal

1,5 min Cassandra

```
Menu 10 (Seguimiento GPS) → opcion 1 → HLCU7654321 → Enter → Enter
```

Salen unas 70 posiciones con latitud y longitud reales de Buenos Aires.

> **Decir:** "Las lecturas de sensores son lo que mas crece de todo el sistema: cinco sensores por minuto son 7.200 filas por dia; quinientos serian 720.000. Por eso van en Cassandra, que esta hecha para escribir mucho y repartir en particiones."

> **Y esto, que es lo que lo diferencia:** "La clave de particion incluye el dia. Eso evita que un sensor con anos de historia termine con una particion gigante, pero tiene una consecuencia practica: pedir un rango de fechas son varias consultas, una por dia. Por eso hay un tope de 31 dias."

Acepta el codigo ISO porque es lo que esta impreso en el contenedor real; el UUID tambien funciona. Enter en las fechas usa los ultimos 7 dias.

4. La red de relaciones

1,5 min Neo4j

`Menu 9 (Consultas de la red) → opcion 2: Envios y operadores de un cliente`

Pedi el ID de **AgroGlobal**. Si no lo tenes a mano, salí a *Menu 1 (Clientes) → opcion 2: Listar todos* y copialo.

> **Decir:** "Esta pregunta en SQL seria una cadena de JOIN: cliente, envio, tramo, vehiculo, operador. En un grafo es recorrer relaciones, que es su operacion natural."

> **Lo importante, y conviene decirlo sin que lo pregunten:** "Neo4j no es duenio de nada. Todo lo que tiene esta derivado de PostgreSQL. Si se borrara entero, hay una opcion de menu que lo reconstruye. Eso es lo que hace defendible que su consistencia sea eventual."

5. El Outbox

2 min PostgreSQL → Neo4j + Cassandra

Esta es la parada mas importante. Si solo te da el tiempo para una, que sea esta.

`Menu 16 (Sincronizacion) → opcion 2: Verificar consistencia`

Responde `TODO CONSISTENTE`.

> **Decir:** "Cuando se crea un envio hay que guardarlo en PostgreSQL y reflejarlo en el grafo. Son dos bases: no comparten transaccion. Si escribis en las dos, una despues de la otra, y la segunda falla, te queda un envio que el grafo no tiene y **nadie se entera**. Eso se llama problema de la doble escritura."

> **La solucion:** "Escribimos solo en PostgreSQL: el dato y un evento en una tabla `outbox`, en la misma transaccion. Despues un worker lee los eventos pendientes y los aplica en Neo4j y Cassandra, con reintentos. Convertimos **dos escrituras que podian desincronizarse en una sola escritura transaccional mas una propagacion reintentable**."

> **Y el detalle que demuestra que lo pensaste:** "Como un evento se puede reintentar, el proyector tiene que ser idempotente. Por eso las consultas Cypher usan `MERGE` y nunca `CREATE`. Reintento mas idempotencia da el efecto de 'exactamente una vez'."

`Menu 16 → opcion 3: Reconstruir el grafo desde PostgreSQL`

Corrrelo si hay tiempo: borra Neo4j y lo rehace. Es la red de seguridad, y se puede hacer justamente porque el grafo no es duenio de nada.

6. El promedio que ninguna base responde sola

2 min MongoDB + PostgreSQL

`Menu 14 (Incidentes) → opcion 7: Promedio → eje 2 (ESTADO)`

Sale una tabla con los cinco estados de envio, sus incidentes, sus envios y el promedio.

> **Decir:** "Los incidentes estan en MongoDB y los envios en PostgreSQL. MongoDB cuenta el numerador y PostgreSQL el denominador. El promedio lo compone el servicio, porque ninguna consulta sola lo responde."

> **Y aca va lo mejor del TP:** "El caso del estado es el mas interesante. El estado del envio **no esta en el documento del incidente, y no puede estarlo**: el incidente guarda una foto de cuando ocurrio, y el estado del envio cambia con el tiempo. Entonces MongoDB agrupa los incidentes por envio, PostgreSQL dice en que estado esta hoy cada uno, y el servicio los suma."

`Menu 14 → opcion 5: Ranking de rutas con mas incidentes`

> **El contraste, que vale la pena marcar:** "Este es el contrario: se resuelve **entero dentro de MongoDB**, con una sola agregacion. Funciona porque cada incidente guarda embebida una foto de la ruta. Una denormalizacion bien elegida convierte una consulta que cruzaria tres motores en una sola."

7. Monitoreo IoT: cuatro motores en una pantalla

2 min Los cinco

`Menu 15 (Monitoreo IoT) → opcion 11: Control de estado de sensores`

Los cuatro sensores activos salen `ACTIVO` con los segundos sin reportar.

> **Decir:** "Un sensor esta inactivo cuando deja de reportar. La pregunta es 'cuanto hace que no escribe', y responderla contra Cassandra obligaria a buscar la ultima lectura de cada sensor recorriendo particiones. En Redis hay un sorted set donde el score es la marca de tiempo del ultimo reporte: es una sola lectura."

`Menu 15 → opcion 12: Cobertura de umbrales`

> **Decir:** "Este control nacio de un error real. Los umbrales cubrian 2 de los 7 tipos de contenedor, asi que la deteccion salteaba sensores **en silencio**. Al completarlos, una corrida identica paso de 14 a 45 alertas. **Una configuracion incompleta es peor que una ausente, porque parece que esta funcionando.**"

Y para cerrar, el cuadro completo:

Dato	Motor	Por que
Ficha del sensor	PostgreSQL	Se relaciona con el contenedor
Lectura	Cassandra	Serie temporal que crece sin parar
Umbral y alerta	MongoDB	Forma variable, detalle embebido
Heartbeat	Redis	Una lectura en vez de un barrido

Si sobra tiempo

Parada	Donde	Que mostrar
Prediccion de atrasos	Menu 12 → opcion 3: Por que un envio es de riesgo	La tabla de motivos y puntos. **Decir:** "no es machine learning a proposito: ante 'por que este envio es ALTO' la unica respuesta seria 'porque lo dijo el modelo'."
Reportes facturables	Menu 13 → opcion 1: Tarifario	Los 11 reportes con su precio. El flujo es pedir, generar y facturar: tres pasos, tres motores.
La cache	Menu 8 → cualquiera, dos veces	La segunda sale de Redis. **Decir:** "si Redis falla, se consulta PostgreSQL igual: un Redis caido hace la app mas lenta, no la rompe."
Deteccion en vivo	Menu 15 → opcion 13, dos veces	La segunda da cero alertas nuevas. Es la idempotencia funcionando: el id de una alerta es deterministico.

Datos verificados

Para mostrar	Usar exactamente	Da
Recorrido GPS	`HLCU7654321`	~70 posiciones
GPS alternativo	`MSKU1234567`	~35 posiciones
Movimientos en el grafo	`MSKU1234567`	3 tramos
Lecturas / alertas de un sensor	`SN-IOT-2026-002`	50 lecturas, 25 alertas
Envios demorados	opcion directa	2 de 4 en curso
Prediccion	opcion directa	2 ALTO, 2 BAJO
Alertas abiertas	opcion directa	~100
Cuenta con deuda	Naviera Rio de la Plata	-12.400

Si algo sale vacio o falla

> **No te quedes en silencio arreglandolo.** Deci que estas viendo y por que puede pasar. "Esta vacio porque la ventana por defecto son 7 dias y este sensor no reporto en ese rango" es una respuesta que suma; quedarse callado tocando teclas, no.

Sintoma	Causa probable	Que hacer
"No hay posiciones en ese intervalo"	Los datos IoT envejecieron	Correr el simulador y repetir
Una consulta del grafo da vacio	El Outbox tiene eventos pendientes	Menu 16 → opcion 1: Propagar eventos
No conecta a una base	Contenedor Up pero sin puerto publicado	`docker compose up -d --force-recreate`
"Hay N que coinciden"	El fragmento de ID es ambiguo	Usar mas caracteres, o el codigo / email

Las tres frases para llevarse aprendidas

1. **Sobre el Outbox:** "Convierte dos escrituras que podian desincronizarse en una sola escritura transaccional mas una propagacion reintentable."

2. **Sobre por que varias bases:** "Cada dato esta donde su forma lo pide. Y lo que perdimos es la transaccion que abarca todo: por eso existe el Outbox y por eso las validaciones entre motores las hace el servicio."
3. **Sobre el promedio por estado:** "Ese dato no esta ni puede estar en el documento: el incidente guarda una foto y el estado del envio cambia. Lo responden las dos bases juntas."